

Design Document : PostgreSQL semver Extension

1. Structure of the Extension :

When we started this project, the first thing we had to figure out was how PostgreSQL extensions are organized. An extension is not just one file. It is a group of files that each do a different job. Here is what we built and why each file exists.

`semver.c` is the main file of the project. All the C code lives here. Every function that PostgreSQL calls, whether it is reading a version string, comparing two versions, or getting the major number, is written in this file. When we run `make`, this file gets compiled into `semver.so`, which is the plugin file that PostgreSQL loads.

`semver.control` is a small metadata file. PostgreSQL reads this file first when someone types `CREATE EXTENSION semver`. It tells PostgreSQL the name of the extension, its version, and where to find the compiled library. Without this file, PostgreSQL cannot find our extension at all. `semver--1.0.sql` is the registration script. PostgreSQL does not automatically know about the C functions we wrote. We have to introduce them using SQL. This file runs automatically when `CREATE EXTENSION semver` is called. It tells PostgreSQL about the new type, the functions that read and display it, and the operators. Without this file, our C code would compile fine but PostgreSQL would never know it exists.

`Makefile` handles the build process. We did not write a complicated build script. We used PostgreSQL's own build system called `PGXS`, which only needed four lines of settings from us. It knows where all the PostgreSQL headers are, what compiler flags to use, and where to copy the final files.

`sql/semver_test.sql` and `expected/semver_test.out` are the test files. The test script has SQL queries. The expected output file has exactly what those queries should print. PostgreSQL's test runner compares them line by line.

2. The Role of the Magic Block

PostgreSQL is careful about which libraries it loads. Before loading any extension, it checks whether the library was compiled for the same version of PostgreSQL that is currently running. This check happens through something called the magic block:
`PG_MODULE_MAGIC;`

This one line goes at the top of `semver.c`. It puts version information into our compiled library. When PostgreSQL loads `semver.so`, it reads this information and checks it against its own version. If they match, the loading continues. If they do not match, PostgreSQL refuses to load the library and shows an error.

There are two rules. First, it must appear in exactly one file. If we forget it, PostgreSQL will not load. If we put it in two files, the compiler gives a duplicate symbol error. We put it at the top of `semver.c` and did not touch it again.

3. The Role of PGXS

When we compile normal C code, we have to tell the compiler where header files are, what flags to use, and where to put the output. PostgreSQL has many internal header files in many folders. Getting all the compiler settings right by hand would be very difficult.

PGXS solves this. It is a Makefile file that comes with PostgreSQL's development package. Our entire Makefile only has four variable settings:

```
MODULE_big = semver
```

```
OBJS = semver.o
```

```
EXTENSION = semver
```

```
DATA = semver--1.0.sql
```

After that, we include PGXS and it handles everything. When we run `make`, PGXS finds the right include paths, the right compiler flags, and the right output format. When we run `sudo make install`, PGXS copies `semver.so` to PostgreSQL's library folder and copies `semver.control` and `semver--1.0.sql` to PostgreSQL's extension folder.

Four lines of settings and the whole build system works. The `pg_config` tool that PGXS uses internally knows everything about the current PostgreSQL installation on our machine.

4. The In-Memory Representation of the Type

When PostgreSQL stores a `semver` value, either in a table or as a result during a query, it needs a place to put it in memory. We made a simple C struct for this:

```
typedef struct {  
    int32 major;  
    int32 minor;  
    int32 patch;  
} Semver;
```

Each field is an `int32`, which is a 32-bit integer. There are three fields and each takes 4 bytes, so the struct is exactly 12 bytes total. This is why we wrote `INTERNALLENGTH = 12` in the SQL type definition. We are telling PostgreSQL exactly how much space each `semver` value needs. When someone inserts `'16.4.0'` into a `semver` column, this is what happens in memory:

```
+-----+-----+-----+  
| major = 16 | minor = 4 | patch = 0 |  
| (4 bytes) | (4 bytes) | (4 bytes) |  
+-----+-----+-----+  
Total = 12 bytes
```

PostgreSQL saves these 12 bytes to disk. When it needs to show the value, it reads the 12 bytes back, rebuilds the struct, and calls our `semver_out` function which writes it back as `"16.4.0"`.

5. The Comparison Helper

The most important function in our extension is one that is never called directly from SQL. It is a private C helper called `semver_compare`:

```
static int semver_compare(Semver *a, Semver *b)
{
    if (a->major != b->major)
        return a->major - b->major;
    if (a->minor != b->minor)
        return a->minor - b->minor;
    return a->patch - b->patch;
}
```

This function is the main reason the extension exists. It compares two semver values field by field using integer math, not string comparison. The return value follows the standard C rule: negative means a is less than b, zero means they are equal, positive means a is greater than b. The important test case is 1.10.0 versus 1.9.0. String comparison gets this wrong because it compares character by character and thinks 1.10.0 comes before 1.9.0 alphabetically. Our function compares the minor fields as integers: 10 is greater than 9, so 1.10.0 is correctly seen as the larger version.

All six comparison operators call this one helper. The comparison logic lives in one place. If we ever need to fix a bug, we fix it once and all six operators get the fix automatically.

6. The SQL Registration Pattern

Writing the C functions is only half the work. PostgreSQL also needs to know those functions exist, what inputs they take, and what they return. This is done in `semver--1.0.sql`.

Every C function we expose to SQL follows the same pattern. First in the C file:

```
PG_FUNCTION_INFO_V1(semver_lt);
```

```
Datum semver_lt(PG_FUNCTION_ARGS)
```

```
{
    Semver *a = (Semver *) PG_GETARG_POINTER(0);
    Semver *b = (Semver *) PG_GETARG_POINTER(1);
    PG_RETURN_BOOL(semver_compare(a, b) < 0);
}
```

Then in the SQL file:

```
CREATE FUNCTION semver_lt(semver, semver)
    RETURNS bool
    AS 'MODULE_PATHNAME', 'semver_lt'
    LANGUAGE C IMMUTABLE STRICT;
```

MODULE_PATHNAME gets replaced by the actual path to semver.so at runtime. IMMUTABLE tells PostgreSQL this function always gives the same result for the same inputs. STRICT means if either input is NULL, return NULL without calling our code.

For operators we add one more step:

```
CREATE OPERATOR < (  
    LEFTARG    = semver,  
    RIGHTARG   = semver,  
    PROCEDURE  = semver_lt,  
    COMMUTATOR = >,  
    NEGATOR    = >=  
);
```

COMMUTATOR tells the planner that a less than b is the same as b greater than a. NEGATOR tells it that NOT (a less than b) is the same as a greater than or equal to b. These hints help PostgreSQL optimize queries that use our operators.

We followed this same pattern for all six comparison operators and all five utility functions. Once we understood it for one function, the rest were easy to write.

Good point! Let me look at what actual decisions you and Sara made in your code to pick a real one.

Looking at your semver.c, I can see a real design decision you made:

You used sscanf with a %c trick to catch extra characters instead of writing a manual character-by-character parser.

Here is the updated section 7:

7. Design Decision: How We Chose to Parse Version Strings

When we wrote the semver_in function, we had to decide how to read a string like "1.2.3" and turn it into our struct. There were two ways to do this.

Option 1: Write a manual parser We could loop through each character one by one, collect digits, check for dots, and build the numbers ourselves. This gives full control but requires writing around 30 to 40 lines of careful code with many edge cases to handle.

Option 2: Use sscanf with a trick We could use the standard C function sscanf to do the parsing for us. This is what we chose:

```
n = sscanf(str, "%d.%d.%d%c",  
           &result->major,  
           &result->minor,  
           &result->patch,  
           &extra);
```

```
if (n != 3 || result->major < 0 || result->minor < 0 || result->patch < 0)  
    ereport(ERROR, ...);
```

The interesting part is the %c at the end. sscanf returns how many items it successfully read. If someone passes "1.2.3", sscanf reads three integers and stops, returning 3. If someone passes

"1.2.3.4" or "1.2.3abc", the %c catches the extra character and sscanf returns 4. We check that n is exactly 3, which means exactly three integers and nothing else.

We also check that all three fields are not negative, which rejects inputs like "-1.2.3".

Why we chose this approach:

The first reason is simplicity. Our sscanf approach does the same job in 8 lines that a manual parser would do in 40 lines. Less code means fewer chances for bugs.

The second reason is that it already handles everything we need. Our tests confirmed that all of these are correctly rejected:

```
"1.2"      -- only 2 parts, sscanf returns 2, not 3
"1.2.3.4"  -- extra part, %c catches it, sscanf returns 4
"v1.2.3"   -- letter at start, sscanf cannot read first integer
"1.2.x"    -- letter where number expected, sscanf returns 2
""         -- empty string, sscanf returns 0
"-1.2.3"   -- negative, caught by our negativity check
```

The third reason is performance. sscanf is a well-tested standard library function that has been optimized for decades. Writing our own parser would almost certainly be slower and less reliable.

The only trade-off is that sscanf error messages are not very specific. When it fails, we only know the whole string was wrong, not which part. But since our report message shows the full invalid string and our hint explains the correct format, this is perfectly acceptable for a database type error message.